Machine Learning Exam - Comprehensive Question Bank

PART A: INTRODUCTION TO MACHINE LEARNING

Mathematical Questions

Q1. Given a machine learning system with experience E, task T, and performance measure P. If the system's performance improves from 65% to 82% accuracy after processing 10,000 additional samples, calculate:

- a) The performance improvement rate per 1000 samples
- b) If the improvement follows a logarithmic curve $P(n) = a \cdot \log(n) + b$, estimate parameters a and b given $P(1000) = 65\%$ and $P(11000) = 82\%$

Answer:

- a) Improvement = 82% - 65% = 17% over 10,000 samples = 1.7% per 1000 samples
- b) $65 = a \cdot \log(1000) + b$ and $82 = a \cdot \log(11000) + b$
  - Solving: $17 = a \cdot (\log(11000) - \log(1000)) = a \cdot \log(11)$
  - $a = 17/\log(11) \approx 16.36$, $b = 65 - 16.36 \cdot \log(1000) \approx 15.93$

Conceptual Questions

Q2. Explain why Tom Mitchell's definition requires all three components (E, T, P) for machine learning. What happens if any component is missing?

Answer: All three are essential:

- **E (Experience)**: Without data/experience, there's nothing to learn from - it's just hardcoded logic
- **T (Task)**: Without a defined task, there's no objective or direction for learning
- **P (Performance)**: Without measurable performance, we cannot verify if learning occurred or guide improvement Missing any component means either no learning happens or we cannot verify/measure it.

Q3. Differentiate between supervised, unsupervised, and reinforcement learning with respect to:

- Data structure
- Learning objective
- Feedback mechanism

Answer:

Aspect	Supervised	Unsupervised	Reinforcement
Data	(x, y) pairs with labels	Only x, no labels	Sequential states & actions
Objective	Learn f: X→Y mapping	Discover patterns/structure	Maximize cumulative reward
Feedback	Direct (correct labels)	None (self-discovered)	Delayed (reward signals)

Scenario-Based Questions

Q4. You're building a spam email classifier. After training on 5000 emails:

- Training accuracy: 95%
- Validation accuracy: 88%
- Test accuracy: 87%

Your manager asks why we need three separate datasets. Explain the role of each and what the accuracy differences indicate.

Answer:

- **Training Set**: Used to learn parameters. 95% shows model fits training data well.
- **Validation Set**: Used for hyperparameter tuning and model selection. 88% suggests some overfitting (7% drop).
- **Test Set**: Final unbiased evaluation. 87% is true expected performance on unseen data.

The gap between training (95%) and validation/test (87-88%) indicates mild overfitting. We need separate sets to detect this and ensure our model generalizes rather than memorizing.

Intuitive Questions

Q5. Why is clustering considered "unsupervised" when we still provide the number of clusters k? Isn't that a form of supervision?

Answer: Supervision refers to providing **labeled outputs/targets** for each input. In clustering:

- We don't tell the algorithm which samples belong together
- k is a **hyperparameter** (structural choice), not a label
- The algorithm discovers patterns and groupings independently
- It's like saying "organize these into 5 groups" vs "this belongs in group A, that in group B" (supervised)

Q6. In Tom Mitchell's definition, can "performance measure P" ever decrease with more experience E? When might this happen?

Answer: Theoretically no, but practically yes in cases like:

- **Concept drift**: Data distribution changes over time (old experience becomes harmful)
- **Overfitting**: Model memorizes training noise, performance on new data degrades
- **Catastrophic forgetting**: Learning new tasks destroys knowledge of old tasks
- **Adversarial attacks**: Malicious data poisoning degrades performance True learning should improve P, but poor algorithms or changing environments can violate this.

Related Topic Questions

Q7. Compare and contrast Classification vs Regression vs Clustering:

- Output space
- Loss functions typically used
- Evaluation metrics
- Example applications

Answer:

Aspect	Classification	Regression	Clustering
Output	Discrete classes	Continuous values	Group assignments
Loss	Cross-entropy, 0-1 loss	MSE, MAE, Huber	Within-cluster variance
Metrics	Accuracy, F1, AUC	RMSE, R^2, MAE	Silhouette score, Davies-Bouldin
Example	Spam detection	House price	Customer segmentation

PART B: LINEAR REGRESSION

Mathematical Questions

Q8. Derive the normal equation for linear regression starting from the cost function $J(\theta) = (1/2n)\Sigma(h\_\theta(x^{(i)}) - y^{(i)})^2$. Show all steps.

Answer:

$J(\theta) = (1/2n)||X\theta - y||^2$
$\partial J/\partial\theta = (1/n)X^T(X\theta - y) = 0$
$X^T X\theta = X^T y$
$\theta^* = (X^T X)^{(-1)} X^T y$

Q9. Given a dataset with n=4 samples and d=2 features:

$X = [1, 2; 1, 3; 1, 4; 1, 5]$ (with $x_0=1$)
$y = [3; 5; 7; 9]$

Calculate θ^* using the normal equation.

Answer:

$X^T X = [4, 14; 14, 54]$
$X^T y = [24; 86]$

$(X^T X)^{(-1)} = (1/20)[54, -14; -14, 4] = [2.7, -0.7; -0.7, 0.2]$
$\theta^* = [2.7, -0.7; -0.7, 0.2][24; 86] = [4.2 - 60.2; -16.8 + 17.2]$
$\theta^* = [1; 2]$

So $h(x) = 1 + 2x_1$

Q10. For linear regression, prove that the cost function $J(\theta) = (1/2n)\Sigma(\theta^T x^{(i)} - y^{(i)})^2$ is convex.

Answer: A function is convex if its Hessian H is positive semi-definite.

$J(\theta) = (1/2n)(X\theta - y)^T(X\theta - y)$
$\nabla J = (1/n)X^T(X\theta - y)$
$H = \nabla^2 J = (1/n)X^T X$

For any vector v: $v^T H v = (1/n)v^T X^T X v = (1/n)||Xv||^2 \geq 0$
Therefore H is PSD, and $J(\theta)$ is convex.

Conceptual Questions

Q11. Why do we use (1/2n) instead of (1/n) in the cost function? Does it affect the optimal θ^*?

Answer:

- The 1/2 simplifies derivative calculation (cancels with power of 2)
- Doesn't affect θ^* because we're optimizing (finding where derivative = 0)
- Constant factors don't change the location of minimum, only scale the loss value

- Convention for mathematical convenience

Q12. When would (X^T X) be non-invertible? Provide at least 3 scenarios and their implications.

Answer:

1. **Multicollinearity**: Features are linearly dependent (e.g., $x_2 = 2x_1$)
    - Matrix is rank deficient
    - Infinitely many solutions exist
2. **n < d**: More features than samples
    - Underdetermined system
    - Need regularization (Ridge/Lasso)
3. **Duplicate features**: Same column repeated
    - Redundant information
    - Remove duplicates

Implications: Use pseudoinverse, regularization, or feature selection.

Scenario-Based Questions

Q13. You're predicting house prices using linear regression:

- Features: area (500-5000 sq ft), bedrooms (1-5), age (0-100 years)
- After training, you get: θ = [50000, 100, -20000, -500]

Interpret each coefficient and explain if this model makes practical sense.

Answer:

- **θ_0 = 50,000**: Base price (intercept) - a house with 0 area/bedrooms/age costs $50k (unrealistic baseline)
- **θ_1 = 100**: Each sq ft adds $100 (reasonable)
- **θ_2 = -20,000**: Each bedroom **reduces** price by $20k (illogical! Should be positive)
- **θ_3 = -500**: Each year of age reduces price by $500 (reasonable - depreciation)

Problem: Negative coefficient for bedrooms suggests multicollinearity or poor feature scaling. Bedrooms likely correlates with area, causing instability.

Q14. Your linear regression model achieves:

- Training MSE: 0.05
- Test MSE: 4.5

What's happening and how would you address it?

Answer: Problem: Severe overfitting (90x higher test error)

Causes:

- Model too complex for data
- Training set too small
- Features not representative

Solutions:

1. Regularization (Ridge/Lasso)
2. Feature selection/reduction
3. Collect more training data
4. Check for outliers
5. Cross-validation for hyperparameter tuning

Intuitive Questions

Q15. Why is it called "linear" regression when $h_\theta(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_1^2$ can model a parabola?

Answer: "Linear" refers to linearity **in parameters θ**, not in input features:

- h(x) is a **linear combination** of θ values
- Even if x_1^2 is a feature, it's still $\theta_0 \cdot 1 + \theta_1 \cdot x_1 + \theta_2 \cdot x_1^2$
- Each θ appears with power 1 (linear in θ)
- This allows us to use linear algebra (normal equation works)
- Contrast with $\theta_1^{x_1}$ (exponential) - that's non-linear in parameters

Q16. If you have a perfect fit ($J(\theta) = 0$) on training data, is your model guaranteed to perform well on test data? Why or why not?

Answer: No, for several reasons:

1. **Overfitting**: Model memorized training noise, won't generalize
2. **Train/test distribution mismatch**: Test data from different distribution
3. **Small training set**: Perfect fit might be coincidence
4. **High model complexity**: Can always fit n points with degree n-1 polynomial

Perfect training fit often indicates overfitting unless:

- Data is truly deterministic (no noise)
- Model captures true underlying relationship
- Training set is large and representative

Related Topic Questions

Q17. How would you modify linear regression to handle:

- a) Outliers that heavily influence the solution
- b) Multicollinearity between features
- c) Non-linear relationships

Answer: a) **Outliers**:

- Use robust loss functions (Huber loss, MAE instead of MSE)
- Outlier detection and removal
- Weighted least squares

b) **Multicollinearity**:

- Ridge regression (L2 regularization)
- Feature selection/PCA
- Remove redundant features
- Increase training data

c) **Non-linear relationships**:

- Feature engineering (polynomial features, log transforms)
- Kernel methods
- Switch to non-linear models (neural networks, decision trees)

PART C: GRADIENT DESCENT

Mathematical Questions

Q18. For $J(\theta) = (1/2)(\theta_1^2 + 4\theta_2^2)$, perform 3 iterations of gradient descent starting from $\theta = [2, 2]^T$ with $\alpha = 0.1$. Calculate the exact θ values.

Answer:

$\nabla J = [\theta_1; 8\theta_2]$

Iteration 0: $\theta = [2, 2]$
$\nabla J = [2; 16]$
$\theta_1 = 2 - 0.1(2) = 1.8$
$\theta_2 = 2 - 0.1(16) = 0.4$

Iteration 1: $\theta = [1.8, 0.4]$
$\nabla J = [1.8; 3.2]$
$\theta_1 = 1.8 - 0.1(1.8) = 1.62$
$\theta_2 = 0.4 - 0.1(3.2) = 0.08$

Iteration 2: $\theta = [1.62, 0.08]$
$\nabla J = [1.62; 0.64]$
$\theta_1 = 1.62 - 0.1(1.62) = 1.458$
$\theta_2 = 0.08 - 0.1(0.64) = 0.016$

Q19. Derive the update rule for gradient descent in linear regression. Show that:

$\theta_j := \theta_j - \alpha(1/n)\Sigma_i(h\_\theta(x^{(i)}) - y^{(i)})\cdot x_j^{(i)}$

Answer:

$J(\theta) = (1/2n)\Sigma_i(h\_\theta(x^{(i)}) - y^{(i)})^2$

$\partial J/\partial\theta_j = (1/2n)\Sigma_i\ 2(h\_\theta(x^{(i)}) - y^{(i)})\cdot\partial h\_\theta/\partial\theta_j$
$= (1/n)\Sigma_i(h\_\theta(x^{(i)}) - y^{(i)})\cdot x_j^{(i)}$

Update: $\theta_j := \theta_j - \alpha\cdot\partial J/\partial\theta_j$
$= \theta_j - \alpha(1/n)\Sigma_i(h\_\theta(x^{(i)}) - y^{(i)})\cdot x_j^{(i)}$

Q20. For mini-batch SGD with batch size m=32 and dataset size n=1000, calculate:

- a) Number of mini-batches per epoch
- b) Number of updates per epoch
- c) If using 10 epochs, total number of gradient computations

Answer:

- a) 1000/32 = 31.25 → 31 mini-batches (last incomplete batch dropped)

- b) 31 updates per epoch
- c) 10 epochs × 31 updates = 310 gradient computations

Conceptual Questions

Q21. Compare Batch GD, Stochastic GD, and Mini-batch GD on:

- Gradient accuracy
- Convergence stability
- Memory requirements
- Computational speed
- Parallelization capability

Answer:

Aspect	**Batch GD**	**SGD**	**Mini-batch GD**
Gradient accuracy	Exact	Very noisy	Moderately noisy
Convergence	Smooth, deterministic	Oscillates, never settles	Reasonably stable
Memory	High (all n samples)	Low (1 sample)	Medium (m samples)
Speed per update	Slow	Very fast	Fast
Parallelization	Limited	None	Good (GPU-friendly)
Use case	Small datasets	Online learning	Standard choice

Q22. Explain why mini-batch SGD gradient is an unbiased estimator of the true gradient. What does "unbiased" mean and why is it important?

Answer: Unbiased means E[estimate] = true value.

For mini-batch gradient g_B:

$$
\begin{aligned}
E[g_B] &= E[(1/m)\Sigma_{i\in B} \nabla J(\theta, x^{(i)}, y^{(i)})] \\
&= (1/m)\Sigma_{i\in B} E[\nabla J(\theta, x^{(i)}, y^{(i)})] \\
&= (1/m)\cdot m\cdot(1/n)\Sigma_{\text{all } i} \nabla J(\theta, x^{(i)}, y^{(i)}) \\
&= \nabla J(\theta) \quad \text{[true gradient]}
\end{aligned}
$$

Importance:

- On average, we move in the correct direction

- Convergence guarantees hold
- Noise averages out over iterations
- Though variance exists, expectation is correct

Scenario-Based Questions

Q23. You're training a model with 1 million samples. After 1 epoch:

- Batch GD: 2 hours, loss = 0.5
- SGD: 10 minutes, loss = 0.8
- Mini-batch (size=64): 15 minutes, loss = 0.52

Which should you choose and why? Consider you have limited time.

Answer: Choose Mini-batch GD because:

1. **Best trade-off**:
    - Loss (0.52) nearly as good as Batch GD (0.5)
    - 8x faster than Batch GD (15 min vs 2 hours)
    - More stable than SGD (0.52 vs 0.8)
2. **Scalability**:
    - Can train many epochs in Batch GD's 1 epoch time
    - GPU-friendly parallelization
    - After 8 epochs (2 hours), likely beats Batch GD
3. **SGD issues**:
    - Much higher loss (0.8)
    - May never converge to good solution
    - Too noisy for stable training

Q24. Your gradient descent diverges (loss increases). You suspect learning rate. α is currently 0.1. Describe your debugging process.

Answer:

Step 1 - Confirm divergence:

- Plot loss vs iterations
- Check if gradients exploding (print gradient norms)

Step 2 - Test smaller learning rates:

- Try $\alpha = [0.01, 0.001, 0.0001]$
- Use learning rate finder (start small, gradually increase)

Step 3 - Check implementation:

- Verify gradient computation (numerical gradient check)
- Ensure correct update rule ($\theta := \theta - \alpha\nabla J$, not +)
- Check for bugs in loss calculation

Step 4 - Investigate other causes:

- Poor feature scaling (normalize features)
- Exploding gradients (gradient clipping)
- Bad initialization (Xavier/He initialization)
- Non-convex with bad local minimum

Step 5 - Solutions:

- Learning rate schedule (decay over time)
- Adaptive optimizers (Adam, RMSprop)
- Gradient clipping
- Better preprocessing

Intuitive Questions

Q25. If learning rate is "too large," loss might increase. If "too small," training is slow. Can you be more precise about what "too large" and "too small" mean mathematically?

Answer:

Too Large ($\alpha > 2/L$ where L is Lipschitz constant):

- Step size exceeds distance to minimum
- Overshoots and bounces back further
- For quadratic $J(\theta) = (1/2)\theta^T A \theta$, need $\alpha < 2/\lambda_{max}(A)$
- Symptoms: loss oscillates or increases

Too Small ($\alpha \ll$ optimal):

- Makes progress but inefficiently
- Takes many iterations: θ moves by tiny increments
- Time to convergence ~ $1/\alpha$
- Symptoms: very slow training, gets stuck

Goldilocks zone:

- For strongly convex: $\alpha \approx 1/L$ gives linear convergence
- Adaptive methods (Adam) automatically adjust
- Rule of thumb: start at 0.001-0.01, tune by factor of 10

Q26. Why does SGD sometimes escape local minima better than batch GD, even though it's noisier?

Answer:

Noise as a feature:

1. **Stochasticity adds exploration**: Random fluctuations can push out of shallow local minima
2. **Like simulated annealing**: Random jumps help escape bad regions
3. **Smoother loss landscape**: Batch GD sees true (possibly non-convex) loss; SGD sees noisy approximation that might "blur out" small local minima

Batch GD problems:

- Deterministic path gets stuck in local minima
- No mechanism to escape once gradient ≈ 0
- Needs random restarts or momentum

However:

- Modern deep learning: local minima often not a problem (loss landscape is well-behaved)
- Noise helps early training, hurts final convergence
- Solution: Start noisy (SGD), then reduce noise (larger batches) or learning rate

Related Topic Questions

Q27. How do momentum-based optimizers (SGD with momentum, Nesterov) differ from vanilla gradient descent? Explain the intuition and update rules.

Answer:

Vanilla GD: $\theta := \theta - \alpha\nabla J$

- Only current gradient matters
- Changes direction instantly

SGD with Momentum:

$$v := \beta v + \nabla J$$
$$\theta := \theta - \alpha v$$

- **Intuition**: Rolling ball accumulates velocity
- **Benefits**:
  - Accelerates in consistent directions
  - Dampens oscillations
  - Escapes plateau regions faster
- **$\beta \in [0,1]$**: typically 0.9, controls memory of past gradients

Nesterov Momentum:

$$v := \beta v + \nabla J(\theta - \alpha\beta v) \quad \text{[look-ahead gradient]}$$
$$\theta := \theta - \alpha v$$

- **Intuition**: Look ahead before computing gradient (corrective behavior)
- **Benefits**: More responsive to changes, better convergence
- **Anticipatory**: Reduces overshooting

Comparison:

- Momentum: "Move first, look later"
- Nesterov: "Look first, then move"
- Both: Dramatically faster than vanilla GD on ravines/valleys

---

PART D: LOGISTIC REGRESSION

Mathematical Questions

Q28. Derive the sigmoid function starting from odds and log-odds. Show why $\sigma(z) = 1/(1+e^{-z})$ is the natural

choice for binary classification.

Answer:

Given: p = P(y=1|x), q = 1-p = P(y=0|x)

Odds: p/q = p/(1-p)
Log-odds: log(p/(1-p)) = z = θ^T x [assume linear]

Solving for p:
p/(1-p) = e^z
p = (1-p)e^z
p = e^z - pe^z
p(1 + e^z) = e^z
p = e^z/(1+e^z) = 1/(1+e^(-z)) = σ(z)

Why natural:
- Maps R → (0,1) smoothly
- Symmetric: σ(-z) = 1 - σ(z)
- Nice derivative: σ'(z) = σ(z)(1-σ(z))
- Probabilistic interpretation via logistic distribution

Q29. Prove that the derivative of sigmoid is σ'(z) = σ(z)(1-σ(z)).

Answer:

σ(z) = 1/(1+e^(-z))

Using quotient rule or rewrite:
σ(z) = (1+e^(-z))^(-1)

σ'(z) = -(1+e^(-z))^(-2) · (-e^(-z))
= e^(-z)/(1+e^(-z))²

Factor out from numerator and denominator:
= (e^(-z)/(1+e^(-z))) · (1/(1+e^(-z)))
= (1 - 1/(1+e^(-z))) · 1/(1+e^(-z))
= (1 - σ(z)) · σ(z)
= σ(z)(1 - σ(z)) ✓

Q30. For logistic regression, derive the gradient of negative log-likelihood:

$L(\theta) = -\Sigma_i[y^{(i)}\log(h_\theta(x^{(i)})) + (1-y^{(i)})\log(1-h_\theta(x^{(i)}))]$

Show that $\partial L/\partial\theta_j = \Sigma_i(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$

Answer:

For single sample:

$l = -[y \cdot \log(\sigma(z)) + (1-y) \cdot \log(1-\sigma(z))]$ where $z = \theta^T x$

$$\begin{aligned}
\partial l/\partial z &= -[y \cdot \sigma'(z)/\sigma(z) - (1-y) \cdot \sigma'(z)/(1-\sigma(z))] \\
&= -[y \cdot \sigma(1-\sigma)/\sigma - (1-y) \cdot \sigma(1-\sigma)/(1-\sigma)] \\
&= -[y(1-\sigma) - (1-y)\sigma] \\
&= -[y - y\sigma - \sigma + y\sigma] \\
&= -(y - \sigma) \\
&= \sigma(z) - y \\
&= h_\theta(x) - y
\end{aligned}$$

$\partial l/\partial \theta_j = (\partial l/\partial z)(\partial z/\partial \theta_j) = (h_\theta(x) - y) \cdot x_j$

For all samples:

$\partial L/\partial \theta_j = \Sigma_i (h_\theta(x^{(i)}) - y^{(i)}) \cdot x_j^{(i)}$

Conceptual Questions

Q31. Why do we maximize likelihood instead of minimizing error rate (0-1 loss) in logistic regression?

Answer:

Problems with 0-1 loss:

1. **Non-differentiable**: Gradient is 0 everywhere or undefined
   - Can't use gradient descent
   - No information about "how wrong" we are
2. **Non-convex**: Many local minima
3. **Discrete jumps**: Small parameter changes don't change loss until prediction flips

Benefits of log-likelihood:

1. **Smooth and differentiable**: Nice gradients for optimization
2. **Convex**: Guaranteed global minimum
3. **Probabilistic interpretation**: Maximum Likelihood Estimation principle
4. **Surrogate loss**: Approximates 0-1 loss but optimizable
5. **Calibrated probabilities**: Not just classification, but confidence

Connection: Minimizing negative log-likelihood ≈ minimizing classification error, but with better optimization properties.

Q32. Explain the difference between Maximum Likelihood Estimation (MLE) and Maximum A Posteriori (MAP) estimation. When is MAP better?

Answer:

MLE: $\theta_{MLE} = \text{argmax } P(D|\theta)$

- Frequentist approach
- θ is fixed but unknown
- Only uses data likelihood
- No prior beliefs

MAP: $\theta_{MAP} = \text{argmax } P(\theta|D) = \text{argmax } P(D|\theta)P(\theta)$

- Bayesian approach
- θ is random variable
- Combines likelihood × prior
- Incorporates prior knowledge

When MAP is better:

1. **Small datasets**: Prior prevents overfitting
2. **Prior knowledge exists**: Use domain expertise
3. **Regularization**: Prior acts as regularizer (e.g., Gaussian prior → L2 regularization)
4. **Uncertainty quantification**: Full posterior distribution

Example:

- MLE: Flip coin 3 times, get 3 heads → $\theta_{MLE} = 1.0$ (unrealistic)
- MAP: With prior Beta(2,2) → $\theta_{MAP} \approx 0.71$ (more reasonable)

Scenario-Based Questions

Q33. You trained a logistic regression model for disease diagnosis. For a patient, your model outputs $h_\theta(x) = 0.55$. Your colleague says "It's positive since >0.5" but the doctor says "Too uncertain, run more tests." Who's right and why?

Answer:

Both perspectives valid:

Colleague (mathematical):

- Threshold 0.5 maximizes accuracy when classes balanced
- $0.55 > 0.5 \rightarrow$ predict positive
- Technically correct classification rule

Doctor (practical):

- Medical decisions have asymmetric costs
- False negative (miss disease): patient dies
- False positive (wrong diagnosis): unnecessary treatment
- 0.55 is only 5% confidence margin!

Better approach:

1. **Adjust threshold**: Use 0.3 or 0.4 for medical diagnosis (favor sensitivity over specificity)
2. **Confidence intervals**: $0.55 \pm$ uncertainty estimate
3. **Cost-sensitive learning**: Weight false negatives higher during training
4. **Risk analysis**: Expected cost = $P(FN) \times Cost(FN) + P(FP) \times Cost(FP)$

Answer: In high-stakes domains, 0.55 is too uncertain. Should adjust threshold or get more features.

Q34. Your binary classifier achieves:

- Training accuracy: 99%
- Test accuracy: 98%
- But data is 95% class 0, 5% class 1

Is this a good model? How would you properly evaluate it?

Answer:

No, poor model!

Problem: Baseline (always predict class 0) gives 95% accuracy. Model only 3-4% better than doing nothing.

Proper evaluation:

1. **Confusion Matrix**:

```
          Predicted
         0      1
Actual 0 [TN]   [FP]
       1 [FN]   [TP]
```

2. **Better metrics**:
    - **Sensitivity/Recall**: TP/(TP+FN) - how many actual positives caught
    - **Specificity**: TN/(TN+FP) - how many actual negatives caught
    - **Precision**: TP/(TP+FP) - of predicted positives, how many correct
    - **F1-Score**: Harmonic mean of precision and recall
    - **AUC-ROC**: Area under ROC curve (threshold-independent)
3. **Class-specific accuracy**:
    - Accuracy on class 0 samples
    - Accuracy on class 1 samples (likely very poor)
4. **Actions**:
    - Rebalance training data (oversampling minority, undersampling majority)
    - Use class weights in loss function
    - Change threshold (maybe predict 1 if h(x) > 0.3)
    - Try more sophisticated models

Intuitive Questions

Q35. Why does logistic regression assume a linear decision boundary when the sigmoid is clearly non-linear?

Answer:

Key insight: Decision boundary is where P(y=1) = P(y=0) = 0.5

```
h_θ(x) = σ(θ^T x) = 0.5
⟺ θ^T x = 0  [since σ(0) = 0.5]
```

This is a **hyperplane** (linear)!

Non-linearity of sigmoid:

- Maps θ^T x → probabilities (0,1)
- Provides smooth probability estimates
- Makes loss function differentiable

But decision boundary:

- Just the set $\{x : \theta^T x = 0\}$
- Completely linear in x
- Sigmoid doesn't curve the boundary, just the probabilities

To get non-linear boundaries:

- Add polynomial features: x_1, x_2, x_1^2, x_1x_2, x_2^2
- Use kernel methods
- Neural networks (later topic)

Q36. The perceptron uses a hard threshold (0 or 1) while logistic regression uses sigmoid. Why does logistic regression train better?

Answer:

Perceptron issues:

1. **No gradient**: Output jumps 0→1, derivative is 0 or undefined
2. **Binary feedback**: "Right" or "wrong" but no sense of "how wrong"
3. **Updates only on mistakes**: Ignores correctly classified points
4. **No probabilistic output**: Can't express uncertainty

Logistic regression advantages:

1. **Smooth gradients**: $\sigma'(z) = \sigma(z)(1-\sigma(z))$ always defined
2. **Continuous feedback**: Loss decreases gradually as confidence improves
3. **All points contribute**: Even correct predictions update weights (toward higher confidence)
4. **Probability calibration**: Outputs meaningful probabilities
5. **Principled optimization**: MLE framework

When perceptron works:

- Linearly separable data
- Speed critical (simpler updates)
- Don't need probabilities

Related Topic Questions

Q37. Compare Linear Regression vs Logistic Regression systematically:

Answer:

Aspect	Linear Regression	Logistic Regression
Task	Regression	Binary Classification
Output	$\mathbb{R}$ (continuous)	[0,1] (probability)
Hypothesis	$h(x) = \theta^T x$	$h(x) = \sigma(\theta^T x)$
Loss	MSE: $(h(x)-y)^2$	NLL: $-[y \log h + (1-y)\log(1-h)]$
Assumptions	Linear relationship	Linear decision boundary
Optimization	Closed form or GD	Gradient ascent/descent
Interpretation	Change in y per unit x	Log-odds ratio

Q38. How would you extend binary logistic regression to multi-class classification (K classes)?

Answer:

Approach 1: One-vs-Rest (OvR)

- Train K binary classifiers
- Classifier i: class i vs all others
- Predict: $\text{argmax}_i h_i(x)$
- Issues: Probabilities don't sum to 1

Approach 2: Softmax Regression (Multinomial Logistic)

- K output units, one per class
- Softmax: $P(y=k|x) = \exp(\theta_k^T x) / \Sigma_j \exp(\theta_j^T x)$
- Loss: Cross-entropy
- Proper probability distribution: $\Sigma_k P(y=k|x) = 1$
- Gradient: $(h_k(x) - y_k) \cdot x$ where y_k is one-hot encoded

Approach 3: One-vs-One

- Train $K(K-1)/2$ classifiers (each pair)

- Voting scheme
- Computationally expensive

Best choice: Softmax regression (used in neural networks)

PART E: PROBABILITY & PARAMETER ESTIMATION

Mathematical Questions

Q39. Given a coin with unknown bias θ. You flip it 10 times and get 7 heads. Calculate:

- a) MLE estimate θ_MLE
- b) MAP estimate θ_MAP using Beta(2,2) prior
- c) Posterior distribution

Answer:

a) MLE:

$L(\theta) = \theta^k (1-\theta)^{(n-k)} = \theta^7 (1-\theta)^3$
$\log L = 7 \log \theta + 3 \log(1-\theta)$
$d/d\theta [\log L] = 7/\theta - 3/(1-\theta) = 0$
$7(1-\theta) = 3\theta$
$\theta_{MLE} = 7/10 = 0.7$

b) MAP with Beta(2,2) prior:

Prior: $P(\theta) \propto \theta^{(\alpha-1)}(1-\theta)^{(\beta-1)} = \theta^1(1-\theta)^1$
Posterior $\propto$ Likelihood × Prior
$\propto \theta^7(1-\theta)^3 \cdot \theta^1(1-\theta)^1$
$\propto \theta^8(1-\theta)^4$

Mode of Beta(α,β): $(\alpha-1)/(\alpha+\beta-2)$
$\theta_{MAP} = (9-1)/(9+5-2) = 8/12 = 0.667$

c) Posterior: Beta(α+k, β+n-k) = Beta(2+7, 2+3) = Beta(9,5)

Q40. Prove that Beta distribution is a conjugate prior for Bernoulli likelihood. Show the posterior has the same family.

Answer:

Prior: $\theta \sim \text{Beta}(\alpha, \beta)$

$P(\theta) = [\Gamma(\alpha+\beta)/(\Gamma(\alpha)\Gamma(\beta))]\ \theta^{(\alpha-1)}(1-\theta)^{(\beta-1)}$

Likelihood: k heads in n flips

$P(D|\theta) = \theta^{k}(1-\theta)^{(n-k)}$

Posterior:

$P(\theta|D) \propto P(D|\theta)P(\theta)$

$\propto \theta^{k}(1-\theta)^{(n-k)} \cdot \theta^{(\alpha-1)}(1-\theta)^{(\beta-1)}$

$\propto \theta^{(k+\alpha-1)}(1-\theta)^{(n-k+\beta-1)}$

This is $\text{Beta}(\alpha+k, \beta+n-k)$ ✓

Conjugacy: Prior and posterior are same family (Beta)

Q41. For a Gaussian likelihood with known variance σ^2 and Gaussian prior on mean μ:

Likelihood: $P(x|\mu) = N(\mu, \sigma^2)$

Prior: $P(\mu) = N(\mu_0, \sigma_0^2)$

Derive the posterior $P(\mu|x)$.

Answer:

Posterior is also Gaussian (conjugate prior):

$P(\mu|x) = N(\mu_n, \sigma_n^2)$

where:

Precision (inverse variance):

$\tau = 1/\sigma^2$, $\tau_0 = 1/\sigma_0^2$

Posterior precision:

$\tau_n = \tau_0 + n\tau$

Posterior mean:

$\mu_n = (\tau_0 \mu_0 + n\tau \bar{x})/\tau_n$

$= (\sigma^2\mu_0 + n\sigma_0^2\bar{x})/(\sigma^2 + n\sigma_0^2)$

Weighted average of prior mean and data mean!

More data → posterior closer to $\bar{x}$

Stronger prior → posterior closer to μ_0

Conceptual Questions

Q42. Explain the fundamental philosophical difference between Frequentist and Bayesian approaches. Give an example where they give different answers.

Answer:

Frequentist Philosophy:

- Probability = long-run frequency
- Parameters are fixed but unknown constants
- Data is random (from sampling)
- No prior beliefs allowed
- Inference via confidence intervals, p-values

Bayesian Philosophy:

- Probability = degree of belief
- Parameters are random variables
- Data is observed (fixed after collection)
- Prior beliefs explicitly modeled
- Inference via posterior distribution

Example - Coin Flipping:

Scenario: Flip coin 3 times, get 3 heads. What's the probability of heads?

Frequentist:

- $\theta\_MLE = 3/3 = 1.0$
- "If we repeated this experiment many times..."
- 95% CI: [0.29, 1.0] (wide due to small sample)
- No prior knowledge used

Bayesian:

- With uniform prior Beta(1,1): $E[\theta] = 4/5 = 0.8$
- With informed prior Beta(5,5): $E[\theta] \approx 0.615$
- "Given what we've seen and what we knew before..."
- Posterior distribution quantifies all uncertainty

Key difference: Bayesian shrinks toward prior (regularization effect), Frequentist takes data at face value.

Q43. What is a conjugate prior and why is it useful? Give 3 examples from the conjugate prior table.

Answer:

Definition: A prior is conjugate to a likelihood if the posterior has the same distributional family as the prior.

Benefits:

1. **Analytical tractability**: Posterior has closed form (no integration needed)
2. **Computational efficiency**: Update by simple parameter adjustment
3. **Interpretable**: Parameters have clear meaning (pseudo-counts)
4. **Sequential updating**: Easy to incorporate new data

Examples:

1. **Beta-Bernoulli**:
    - Likelihood: Bernoulli(θ)
    - Prior: Beta(α, β)
    - Posterior: Beta(α+k, β+n-k)
    - Use: Coin flips, click-through rates
2. **Gamma-Poisson**:
    - Likelihood: Poisson(λ)
    - Prior: Gamma(α, β)
    - Posterior: Gamma($\alpha+\Sigma x\_i$, β+n)
    - Use: Count data, rare events
3. **Gaussian-Gaussian**:
    - Likelihood: N(μ, σ^2) [known σ^2]
    - Prior: N(μ_0, σ_0^2)
    - Posterior: N(updated mean, updated variance)
    - Use: Sensor measurements, heights

Q44. In Bayesian inference, why do we need the evidence term P(D) in Bayes' theorem? Can we ignore it?

Answer:

Bayes Theorem: $P(\theta|D) = P(D|\theta)P(\theta)/P(D)$

Role of P(D):

- Normalization constant (ensures posterior sums/integrates to 1)

- $P(D) = \int P(D|\theta)P(\theta)\, d\theta$ (marginal likelihood)

When we can ignore it:

1. **MAP estimation**: argmax $P(\theta|D)$ = argmax $P(D|\theta)P(\theta)$
    - $P(D)$ doesn't depend on θ, doesn't affect maximum
2. **Conjugate priors**: Proportionality is enough
    - We know the family, just need to match parameters
3. **MCMC sampling**: Sample from unnormalized posterior
    - Normalization not needed for Metropolis-Hastings

When we can't ignore it:

1. **Full posterior distribution**: Need proper probabilities
2. **Model comparison**: $P(D)$ = evidence = model quality
3. **Predictive distributions**: $P(x_{new}|D)$ requires integration over posterior
4. **Bayesian hypothesis testing**: Bayes factors use evidence

Practical issue: $P(D)$ often intractable (high-dimensional integral) → approximate inference methods needed

Scenario-Based Questions

Q45. You're estimating click-through rate (CTR) for a new ad. After 100 impressions:

- Ad A: 5 clicks (5%)
- Ad B: 2 clicks (2%)

Your manager wants to show Ad A. But you know Ad B is from a trusted advertiser. How does Bayesian approach help?

Answer:

Frequentist (MLE):

- $CTR_A = 5/100 = 5\%$
- $CTR_B = 2/100 = 2\%$
- Choose Ad A ✓

Bayesian with Priors:

Ad A (unknown new advertiser):

- Prior: Beta(1,1) [uniform, no information]
- Posterior: Beta(1+5, 1+95) = Beta(6, 96)
- E[CTR_A] = 6/102 ≈ 5.88%
- 95% credible interval: [2.4%, 11%]

Ad B (trusted, historically 8% CTR):

- Prior: Beta(8, 92) [encodes historical 8%]
- Posterior: Beta(8+2, 92+98) = Beta(10, 190)
- E[CTR_B] = 10/200 = 5%
- 95% credible interval: [2.8%, 8.1%]

Analysis:

- Both have similar expected CTR (~5-6%)
- Ad B has **much more certainty** (narrower interval)
- Ad A could be anywhere from 2-11%
- With only 100 samples, data is noisy

Decision:

- Risk-averse: Choose Ad B (more predictable)
- Risk-seeking: Choose Ad A (higher upside)
- **Best**: Thompson Sampling (explore/exploit trade-off)

Q46. You're diagnosing a rare disease (1% prevalence). Your test has 95% sensitivity and 90% specificity. A patient tests positive. What's the probability they have the disease?

Answer:

Given:

- P(Disease) = 0.01
- P(Pos|Disease) = 0.95 (sensitivity)
- P(Neg|No Disease) = 0.90 (specificity) → P(Pos|No Disease) = 0.10

Bayes' Theorem:

P(Disease|Pos) = P(Pos|Disease)P(Disease) / P(Pos)

P(Pos) = P(Pos|Disease)P(Disease) + P(Pos|No Disease)P(No Disease)
= 0.95×0.01 + 0.10×0.99
= 0.0095 + 0.099
= 0.1085

P(Disease|Pos) = (0.95×0.01) / 0.1085
= 0.0095 / 0.1085
≈ 8.76%

Interpretation:

- Despite positive test, only 8.76% chance of disease!
- **Base rate fallacy**: Rare disease → most positives are false alarms
- 10% false positive rate on 99% healthy population overwhelms signal

Clinical implications:

- Don't diagnose on single test
- Run confirmatory tests
- Consider patient history (update prior)

Intuitive Questions

Q47. If you flip a fair coin 100 times and get 100 heads, would a Bayesian still believe the coin is fair? Why or why not?

Answer:

Depends on prior strength!

Weak prior (e.g., Beta(2,2)):

Posterior: Beta(2+100, 2+0) = Beta(102, 2)
E[θ] = 102/104 ≈ 0.98

Basically believes coin is biased toward heads.
Data overwhelms weak prior.

Strong prior (e.g., Beta(1000,1000) - very confident in fairness):

Posterior: Beta(1000+100, 1000+0) = Beta(1100, 1000)
$E[\theta] = 1100/2100 \approx 0.524$

Still close to 0.5! Prior dominates.
Would need much more data to overcome.

Realistic prior (e.g., Beta(10,10)):

Posterior: Beta(110, 10)
$E[\theta] = 110/120 \approx 0.917$

Shifted significantly, but not all the way to 1.0

Key insight:

- Bayesian inference **never** completely abandons prior
- Strong evidence can overcome prior, but gradually
- This is **feature not bug**: prevents overfitting to anomalous data
- 100 heads is extremely unlikely if truly fair ($p \approx 7.9 \times 10^{-31}$)
- Rational Bayesian would eventually conclude bias

Q48. Why do we maximize log-likelihood instead of likelihood in MLE? Aren't they the same?

Answer:

They give the same answer (same argmax), but log is **much** better computationally:

Advantages of log:

1. **Numerical stability**:
   - Likelihood: $P(D|\theta) = \prod_i p(x_i|\theta) \rightarrow$ very small numbers (underflow)
   - Example: $(0.1)^{1000} \approx 10^{-1000}$ (can't represent in computer)
   - Log-likelihood: $\Sigma_i \log p(x_i|\theta) \rightarrow$ reasonable numbers
   - Example: $1000 \times \log(0.1) = -2302.6$ (no problem)
2. **Computational efficiency**:
   - Products become sums (easier to compute)
   - Derivatives are simpler (no product rule)
3. **Optimization**:
   - Sums are easier to differentiate than products

- Gradient: $\nabla \log L = \Sigma_i \nabla \log p(x_i|\theta)$

4. **Theoretical convenience**:
    - Log is monotonic: argmax L = argmax log L
    - Connects to information theory (KL divergence)

Example:

$L(\theta) = \theta^3(1-\theta)^7 = 0.000186624$ (hard to work with)
$\log L = 3\log(\theta) + 7\log(1-\theta)$ (clean!)

$d/d\theta [L] = 3\theta^2(1-\theta)^7 - 7\theta^3(1-\theta)^6$ (product rule, messy)
$d/d\theta [\log L] = 3/\theta - 7/(1-\theta)$ (quotient rule, simple!)

Related Topic Questions

Q49. Compare Maximum Likelihood Estimation (MLE), Maximum A Posteriori (MAP), and Full Bayesian Inference:

Answer:

Aspect	**MLE**	**MAP**	**Full Bayesian**
Estimate	Point estimate	Point estimate	Full distribution
Formula	argmax P(D|θ)	argmax P(θ|D)	Compute P(θ|D)
Prior	No prior	Uses prior	Uses prior
Uncertainty	None (just point)	None (just point)	Complete (posterior)
Overfitting	High (small data)	Medium (regularized)	Low (averaging)
Computation	Easy	Easy	Hard (integration)
Prediction	P(y|x, θ_MLE)	P(y|x, θ_MAP)	$\int P(y\|x,\theta)P(\theta\|D)d\theta$

Example - Linear Regression:

- MLE: Minimize MSE → $\theta_{MLE} = (X^TX)^{-1}X^Ty$
- MAP with Gaussian prior: Ridge regression → $\theta_{MAP} = (X^TX + \lambda I)^{-1}X^Ty$
- Full Bayesian: Posterior distribution over θ → predictive distribution

Q50. How does the concept of conjugate priors relate to regularization in machine learning?

Answer:

Deep Connection:

Bayesian view → Regularization:

1. **Ridge Regression** = MAP with Gaussian prior

> Prior: $P(\theta) = N(0, \sigma^2 I)$
> MAP: argmax $[P(D|\theta)P(\theta)]$
> = argmin $[MSE + \lambda||\theta||^2]$
>
> Gaussian prior → L2 regularization!

2. **Lasso** = MAP with Laplace prior

> Prior: $P(\theta) = Laplace(0, b)$
> MAP: argmin $[MSE + \lambda||\theta||_1]$
>
> Laplace prior → L1 regularization!

3. **Elastic Net** = MAP with combined prior

> Prior: Mixture of Gaussian + Laplace
> → L1 + L2 regularization

Conjugacy role:

- Makes posterior tractable
- L2 especially nice: Gaussian-Gaussian conjugacy
- Closed-form solution for Ridge regression

Interpretation:

- **Regularization λ**: Related to prior variance
  - Large λ → Strong prior → Heavy regularization
  - Small λ → Weak prior → Closer to MLE
- **Prior distribution shape**: Type of regularization
  - Gaussian → Smooth penalty (L2)
  - Laplace → Sparse penalty (L1)

PART F: DEEP LEARNING & NEURAL NETWORKS

Mathematical Questions

Q51. For a 2-layer neural network with 3 input units, 4 hidden units, and 2 output units:

- a) How many weight parameters? (including biases)
- b) Write the forward propagation equations
- c) How many computations (multiplications) for one forward pass?

Answer:

a) Parameters:

Layer 1 (input → hidden):
Weights: $3 \times 4 = 12$
Biases: 4
Total: 16

Layer 2 (hidden → output):
Weights: $4 \times 2 = 8$
Biases: 2
Total: 10

Grand Total: $16 + 10 = 26$ parameters

b) Forward Propagation:

$z^{(1)} = W^{(1)}x + b^{(1)}$ (4×1)
$a^{(1)} = \sigma(z^{(1)})$ (4×1)
$z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}$ (2×1)
$\hat{y} = a^{(2)} = \sigma(z^{(2)})$ (2×1)

c) Multiplications:

Layer 1: $3 \times 4 = 12$ multiplications
Layer 2: $4 \times 2 = 8$ multiplications
Total: 20 multiplications per forward pass
(Not counting activation functions and bias additions)

Q52. Prove that a 2-layer neural network with linear activation functions (identity) reduces to a single linear transformation.

Answer:

Given:

$h^{(1)} = W^{(1)}x + b^{(1)}$

$h^{(2)} = W^{(2)}h^{(1)} + b^{(2)}$

Substitute:

$h^{(2)} = W^{(2)}(W^{(1)}x + b^{(1)}) + b^{(2)}$

$= W^{(2)}W^{(1)}x + W^{(2)}b^{(1)} + b^{(2)}$

$= W\_combined\ x + b\_combined$

where:

$W\_combined = W^{(2)}W^{(1)}$

$b\_combined = W^{(2)}b^{(1)} + b^{(2)}$

This is just a single linear layer!

No benefit from multiple layers.

→ Non-linear activations are ESSENTIAL

Q53. Compute the forward and backward pass for this computation graph with chain rule:

$x = 2, w_1 = 3, w_2 = -1$

$z_1 = w_1 * x$

$z_2 = z_1 + w_2$

$z_3 = z_2^2$

Loss $L = z_3$

Find $\partial L/\partial w_1$ and $\partial L/\partial w_2$.

Answer:

Forward Pass:

$z_1 = 3 \times 2 = 6$

$z_2 = 6 + (-1) = 5$

$z_3 = 5^2 = 25$

$L = 25$

Backward Pass:

$\partial L/\partial z_3 = 1$

$\partial L/\partial z_2 = \partial L/\partial z_3 \times \partial z_3/\partial z_2$
$= 1 \times 2z_2$
$= 2 \times 5 = 10$

$\partial L/\partial z_1 = \partial L/\partial z_2 \times \partial z_2/\partial z_1$
$= 10 \times 1 = 10$

$\partial L/\partial w_1 = \partial L/\partial z_1 \times \partial z_1/\partial w_1$
$= 10 \times x$
$= 10 \times 2 = 20$

$\partial L/\partial w_2 = \partial L/\partial z_2 \times \partial z_2/\partial w_2$
$= 10 \times 1 = 10$

Verification (numerical gradient):

$w_1 = 3 + \varepsilon \rightarrow z_1 = (3+\varepsilon)\times 2$, $z_2 = 6+2\varepsilon-1$, $z_3 = (5+2\varepsilon)^2$
$L \approx 25 + 20\varepsilon \rightarrow \partial L/\partial w_1 \approx 20$ ✓

Q54. For ReLU activation, derive the gradient: if f(z) = max(0, z), show f'(z).

Answer:

$f(z) = \max(0, z) = \begin{cases} z & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$

$f'(z) = d/dz\,[f(z)] = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$

Or in indicator notation:
$f'(z) = \mathbb{1}(z > 0)$

At z=0, gradient is technically undefined,
but in practice we set f'(0) = 0 or 1 (doesn't matter much).

Backprop with ReLU:

Forward: a = ReLU(z)
Backward: $\partial L/\partial z = \partial L/\partial a \times \mathbb{1}(z > 0)$

Simple! Just mask gradient where $z \leq 0$.

Conceptual Questions

Q55. Explain why non-linear activation functions are necessary in neural networks. What happens without them?

Answer:

Without non-linearity:

- Stacking linear layers = one linear layer (composition of linear functions is linear)
- Network can only learn linear decision boundaries
- Expressiveness limited to what linear regression/logistic regression can do
- No benefit from depth

With non-linearity:

- Can approximate complex non-linear functions
- Each layer learns hierarchical representations
- Universal approximation theorem applies
- Can solve XOR, spirals, complex patterns

Analogy:

- Linear network = straight line/plane (fixed shape)
- Non-linear network = flexible curve (can bend into any shape)

Types of non-linearity needed:

- Sigmoid: Smooth, bounded (0,1)
- Tanh: Smooth, bounded (-1,1), zero-centered
- ReLU: Simple, addresses vanishing gradient
- Leaky ReLU: Prevents dying neurons

Q56. What is the vanishing gradient problem? Why does it occur with sigmoid activations in deep networks?

Answer:

Problem: Gradients become extremely small in early layers → learning stalls

Why with Sigmoid:

1. **Sigmoid range**: $\sigma(z) \in (0,1)$

2. **Sigmoid derivative**: $\sigma'(z) = \sigma(z)(1-\sigma(z)) \in (0, 0.25]$
    - Maximum at $z=0$: $\sigma'(0) = 0.25$
    - At extremes: $\sigma'(\pm 10) \approx 0$
3. **Chain rule in backprop**: Multiply many derivatives

$$\partial L/\partial W^{(1)} = \partial L/\partial z^{(n)} \times \ldots \times \partial z^{(2)}/\partial z^{(1)} \times \partial z^{(1)}/\partial W^{(1)}$$
$$= \partial L/\partial z^{(n)} \times \sigma'(z^{(n-1)}) \times \ldots \times \sigma'(z^{(1)}) \times x$$

4. **Exponential decay**: Product of many numbers ≤ 0.25
    - 10 layers: $(0.25)^{10} \approx 10^{-6}$
    - Early layers barely update!

Solutions:

- **ReLU**: Derivative is 0 or 1 (no saturation)
- **Batch Normalization**: Keep activations in good range
- **Residual connections**: Gradient highways (skip connections)
- **Better initializations**: Xavier/He initialization
- **Gradient clipping**: Prevent explosion

Q57. Compare activation functions: Sigmoid, Tanh, ReLU, Leaky ReLU. When to use each?

Answer:

Activation	Formula	Range	Derivative	Pros	Cons	Use Case
Sigmoid	$1/(1+e^{-z})$	(0,1)	$\sigma(1-\sigma) \leq 0.25$	Probabilistic output	Vanishing gradient, not zero-centered	Output layer (binary)
Tanh	$(e^{z}-e^{-z})/(e^{z}+e^{-z})$	(-1,1)	$1-\tanh^2 \leq 1$	Zero-centered	Still vanishing gradient	RNNs, hidden layers
ReLU	$\max(0,z)$	$[0,\infty)$	$\mathbb{1}(z>0)$	Simple, no vanishing, sparse	Dying neurons, not zero-centered	Default choice (CNNs, FFNs)
Leaky ReLU	$\max(\alpha z,z)$	$(-\infty,\infty)$	α if $z\leq 0$, 1 if $z>0$	Fixes dying ReLU	Hyperparameter α	When ReLU neurons die

Decision Guide:

- **Output layer**: Sigmoid (binary), Softmax (multi-class), Linear (regression)
- **Hidden layers**: ReLU (default), Leaky ReLU (if dying neurons), Tanh (RNNs)
- **Deep networks**: Avoid sigmoid/tanh (vanishing gradient)

Q58. What is the Universal Approximation Theorem? What are its implications and limitations?

Answer:

Theorem: A feedforward neural network with:

- At least one hidden layer
- Finite number of neurons
- Non-polynomial activation function (e.g., sigmoid, ReLU)

Can approximate any continuous function f: $\mathbb{R}^m \rightarrow \mathbb{R}^n$ on compact domain to arbitrary accuracy.

Mathematical: For any $\varepsilon > 0$, $\exists$ neural network N such that $|N(x) - f(x)| < \varepsilon \ \forall x$

Implications:

- Neural networks are powerful function approximators
- Justifies using NNs for complex tasks
- Single hidden layer is theoretically sufficient

Limitations & Reality:

1. **Doesn't say HOW MANY neurons needed**:
    - Might need exponentially many neurons in shallow network
    - Deep networks are much more efficient (exponential vs polynomial neurons)
2. **Doesn't say HOW TO FIND the weights**:
    - Existence ≠ computability
    - Optimization landscape might be terrible
    - Could have many bad local minima
3. **Doesn't guarantee GENERALIZATION**:
    - Can approximate training data perfectly (memorization)
    - Says nothing about test performance
    - Overfitting is still a problem
4. **Requires sufficient data**:

- To learn complex function, need representative samples
- Curse of dimensionality applies

5. **Practical deep learning**:
    - Uses DEPTH, not width
    - Deep networks learn hierarchical features
    - Depth provides compositionality

Analogy: Knowing a universal Turing machine exists doesn't tell you how to program it efficiently.

Scenario-Based Questions

Q59. You're training a 5-layer neural network for image classification. After 10 epochs:

- Layer 1 weights change by 0.0001%
- Layer 5 weights change by 15%
- Training loss decreasing slowly
- Validation accuracy stuck at 65%

Diagnose the problem and propose solutions.

Answer:

Diagnosis: Vanishing Gradient Problem + Possible Overfitting

Evidence:

1. Layer 1 barely updating (0.0001%) → gradients vanishing
2. Layer 5 updating normally (15%) → gradients strong there
3. Slow training loss decrease → learning inefficient
4. Validation stuck → not improving generalization

Root Causes:

- Sigmoid/Tanh activations saturating
- Poor weight initialization
- Learning rate too small for early layers
- Architecture too deep without skip connections

Solutions:

Immediate fixes:

1. **Switch to ReLU** activation (derivative 0 or 1, no vanishing)
2. **Better initialization**:
    - Xavier for sigmoid/tanh
    - He initialization for ReLU
3. **Increase learning rate** slightly or use adaptive optimizer (Adam)

Architecture improvements: 4. **Batch Normalization**: Normalize activations, helps gradient flow 5. **Residual connections**: Skip connections bypass vanishing gradients 6. **Gradient clipping**: Prevent explosion in deeper layers

Regularization (for stuck validation): 7. **Dropout**: Reduce overfitting 8. **Data augmentation**: More training variability 9. **Early stopping**: Stop before overfitting worsens

Monitoring:

- Plot gradient norms per layer (should be similar magnitude)
- Visualize activation distributions (should be spread out, not saturated)

Q60. Your neural network has these training curves:

- Epoch 1-20: Training loss drops 5.0 → 0.8, Test loss drops 5.0 → 1.2
- Epoch 21-40: Training loss drops 0.8 → 0.1, Test loss increases 1.2 → 2.5

What's happening? What should you do?

Answer:

Diagnosis: Classic Overfitting Pattern

Analysis:

- **Epoch 1-20**: Healthy learning
    - Both train and test improving
    - Test loss higher but following train loss
    - Model learning generalizable patterns
- **Epoch 21-40**: Overfitting kicks in
    - Train loss still decreasing (fitting training noise)
    - Test loss increasing (losing generalization)
    - Gap widening (memorization)

What to do:

1. Immediate action:

- **Early stopping**: Stop at epoch ~20 (best test loss)
- Use saved checkpoint from epoch 20

2. For next training run:

Regularization:

- **Dropout** (0.2-0.5): Randomly drop neurons during training
- **L2 weight decay**: Add $\lambda||W||^2$ to loss (try λ=0.001, 0.01)
- **L1 regularization**: Encourage sparsity

Data:

- **More training data**: Reduces overfitting
- **Data augmentation**: Artificial data expansion (flips, rotations for images)

Architecture:

- **Reduce model capacity**: Fewer layers/neurons
- **Batch normalization**: Acts as regularizer

Training:

- **Validation-based early stopping**: Auto-stop when test loss increases
- **Reduce learning rate**: Slower, more careful learning
- **Ensemble methods**: Average multiple models

3. Diagnostic checks:

```python
# Check model complexity
print(f"Parameters: {model.count_params()}")
print(f"Training samples: {len(train_data)}")
# Rule of thumb: need ~10 samples per parameter
```

Ideal curve: Train and test loss both decrease, staying close together.

Intuitive Questions

Q61. A neural network has 3 layers with 1000 neurons each and ReLU activation. After training, you find 60% of neurons output exactly zero for all inputs. Is this good or bad?

Answer:

Answer: It depends, but likely BAD (Dying ReLU problem)

What's happening:

- 60% of neurons have $z \leq 0$ for ALL inputs
- $\text{ReLU}(z \leq 0) = 0 \rightarrow$ these neurons dead
- Gradient: $\partial \text{ReLU}/\partial z = 0$ when $z \leq 0 \rightarrow$ no learning signal
- Effectively 40% network capacity

Why neurons die:

1. **Poor initialization**: W initialized negative, all $Wx+b < 0$
2. **Large learning rate**: Big update pushes weights to all-negative
3. **Data distribution**: Most inputs lead to negative pre-activations

Is it bad?

- **Sparsity can be good**: Efficient representations, faster computation
- **But 60% is too much**: Wasted capacity, might underfit
- **No recovery**: Dead neurons can never revive (zero gradient)

Healthy sparsity: 10-30% zeros **Concerning**: >50% zeros

Solutions:

1. **Use Leaky ReLU**: $f(z) = \max(0.01z, z) \rightarrow$ always has gradient
2. **Better initialization**: He initialization for ReLU
3. **Lower learning rate**: Prevent drastic weight changes
4. **Batch normalization**: Keep activations in reasonable range
5. **Monitor dead neurons**: Track % zeros during training

Trade-off: Some sparsity is good (regularization), too much is wasteful.

Q62. Why is the derivative of cross-entropy loss with softmax so elegantly simple: (y_hat - y)?

Answer:

The "magical" cancellation:

Setup:

Softmax: $\hat{y}_k = \exp(z_k) / \Sigma_j \exp(z_j)$
Cross-Entropy: $L = -\Sigma_k y_k \log(\hat{y}_k)$

Derivative:

$\partial L/\partial z_i = \Sigma_k (\partial L/\partial \hat{y}_k)(\partial \hat{y}_k/\partial z_i)$

$\partial L/\partial \hat{y}_k = -y_k/\hat{y}_k$

$\partial \hat{y}_k/\partial z_i = \hat{y}_k(\delta_{ki} - \hat{y}_i)$ [Softmax derivative]

Combining:
$\partial L/\partial z_i = \Sigma_k (-y_k/\hat{y}_k) \times \hat{y}_k(\delta_{ki} - \hat{y}_i)$
$= \Sigma_k -y_k(\delta_{ki} - \hat{y}_i)$
$= \Sigma_k -y_k \delta_{ki} + \hat{y}_i \Sigma_k y_k$
$= -y_i + \hat{y}_i \times 1$
$= \hat{y}_i - y_i$ ✓

Why so elegant?

1. **Natural pairing**: Cross-entropy is the "natural" loss for softmax
    - Derived from maximum likelihood
    - Information-theoretic interpretation
2. **Logarithm cancels exponential**:
    - log(exp(z)) terms simplify
    - Partition function normalization cancels
3. **Linear gradient**:
    - Easy to compute
    - Well-behaved (no vanishing/explosion)
    - Intuitive: error = prediction - truth

Contrast with MSE + Sigmoid:

$\partial L/\partial z = (\hat{y} - y) \times \sigma'(z) \times 2$

Extra σ'(z) term causes vanishing gradient!

Lesson: Loss function and output activation should be paired correctly.

Related Topic Questions

Q63. Explain backpropagation algorithm step-by-step for a 2-layer network. Include forward pass, loss computation, and backward pass.

Answer:

Network: Input (x) → Hidden (h) → Output (ŷ)

Given:

- Input: $x \in \mathbb{R}^d$
- Weights: $W^{(1)} \in \mathbb{R}^{d \times h}$, $b^{(1)} \in \mathbb{R}^h$, $W^{(2)} \in \mathbb{R}^{h \times o}$, $b^{(2)} \in \mathbb{R}^o$
- True label: y

Step 1: Forward Pass

```
python

# Layer 1
z^(1) = W^(1)^T x + b^(1)        # (h,)
a^(1) = σ(z^(1))                 # (h,) activation

# Layer 2
z^(2) = W^(2)^T a^(1) + b^(2)    # (o,)
ŷ = σ(z^(2))                     # (o,) prediction

# Loss
L = -[y log(ŷ) + (1-y)log(1-ŷ)]  # scalar
```

Step 2: Backward Pass (Output Layer)

```
python

# Gradient at output
δ^(2) = ∂L/∂z^(2) = ŷ - y        # (o,)

# Gradients for W^(2) and b^(2)
∂L/∂W^(2) = a^(1) ⊗ δ^(2)        # (h, o) outer product
∂L/∂b^(2) = δ^(2)                # (o,)
```

Step 3: Backward Pass (Hidden Layer)

```
python

# Backpropagate through layer 2
δ⁽¹⁾ = (W⁽²⁾ δ⁽²⁾) ⊙ σ'(z⁽¹⁾)  # (h,) element-wise

# Gradients for W⁽¹⁾ and b⁽¹⁾
∂L/∂W⁽¹⁾ = x ⊗ δ⁽¹⁾          # (d, h) outer product
∂L/∂b⁽¹⁾ = δ⁽¹⁾               # (h,)
```

Step 4: Update Weights

```
python

W⁽²⁾ := W⁽²⁾ - α ∂L/∂W⁽²⁾
b⁽²⁾ := b⁽²⁾ - α ∂L/∂b⁽²⁾
W⁽¹⁾ := W⁽¹⁾ - α ∂L/∂W⁽¹⁾
b⁽¹⁾ := b⁽¹⁾ - α ∂L/∂b⁽¹⁾
```

Key Insights:

- **Forward**: Compute activations layer by layer
- **Backward**: Compute gradients in reverse order
- **Chain rule**: $\delta^{(l)} = (W^{(l+1)}\delta^{(l+1)}) \odot \sigma'(z^{(l)})$
- **Store activations**: Need $a^{(l)}$ for gradient computation

Q64. How do different optimizers (SGD, Momentum, Adam) affect training? Compare them conceptually and mathematically.

Answer:

1. Vanilla SGD:

```
θ := θ - α∇J(θ)
```

- **Pros**: Simple, works on convex functions
- **Cons**: Slow on plateaus, overshoots valleys, sensitive to learning rate
- **Use**: Never (better alternatives exist)

2. SGD with Momentum:

```
v := βv + ∇J(θ)        # β ≈ 0.9
θ := θ - αv
```

- **Intuition**: Rolling ball accumulates velocity
- **Pros**: Faster on consistent gradients, dampens oscillations
- **Cons**: Still fixed learning rate
- **Use**: Simple baseline when Adam too complex

3. Nesterov Momentum:

```
v := βv + ∇J(θ - αβv)   # Look ahead!
θ := θ - αv
```

- **Intuition**: Correct course before committing
- **Pros**: Better than momentum, less overshooting
- **Cons**: Slightly more complex
- **Use**: When momentum alone insufficient

4. AdaGrad:

```
cache := cache + (∇J)²   # Accumulate squared gradients
θ := θ - α∇J / (√cache + ε)
```

- **Intuition**: Adapt learning rate per parameter
- **Pros**: Good for sparse features
- **Cons**: Learning rate decays to zero (stops learning)
- **Use**: Rarely (RMSprop/Adam better)

5. RMSprop:

```
cache := β·cache + (1-β)(∇J)²  # Moving average
θ := θ - α∇J / (√cache + ε)
```

- **Intuition**: AdaGrad with decay (doesn't accumulate forever)
- **Pros**: Adaptive per-parameter, doesn't stop learning
- **Cons**: No momentum
- **Use**: Good for RNNs

6. Adam (Adaptive Moment Estimation):

$m := \beta_1 m + (1-\beta_1)\nabla J$ # First moment (momentum)
$v := \beta_2 v + (1-\beta_2)(\nabla J)^2$ # Second moment (RMSprop)
$\hat{m} := m/(1-\beta_1^t)$ # Bias correction
$\hat{v} := v/(1-\beta_2^t)$ # Bias correction
$\theta := \theta - \alpha \cdot \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$ # Update

- **Intuition**: Momentum + RMSprop + bias correction
- **Pros**: Works well out-of-box, adaptive, robust
- **Cons**: More memory, many hyperparameters
- **Use**: **Default choice** for most deep learning
- **Typical**: β_1=0.9, β_2=0.999, α=0.001

Comparison Table:

Optimizer	Speed	Memory	Hyperparameters	Robustness	Use Case
SGD	Slow	Low	1 (α)	Low	Don't use
Momentum	Medium	Low	2 (α, β)	Medium	Simple tasks
RMSprop	Fast	Medium	2 (α, β)	High	RNNs
Adam	Fast	High	4 (α, β_1, β_2, ε)	Very High	**Default**

Practical advice: Start with Adam (α=0.001), only switch if problems arise.

PART G: OPTIMIZATION & TRAINING

Mathematical Questions

Q65. Prove that mini-batch gradient is an unbiased estimator of full batch gradient. Show E[g_batch] = g_full.

Answer:

Setup:

- Full batch gradient: $g_{full} = (1/N)\Sigma_{i=1}^{N} \nabla J(\theta, x_i, y_i)$
- Mini-batch B of size m, sampled uniformly: $g_B = (1/m)\Sigma_{i \in B} \nabla J(\theta, x_i, y_i)$

Proof:

$E[g_B] = E[(1/m)\Sigma_{i \in B} \nabla J(\theta, x_i, y_i)]$

By linearity of expectation:

$= (1/m)\Sigma_{i \in B} E[\nabla J(\theta, x_i, y_i)]$

Since each sample equally likely to be in B:

$= (1/m) \times m \times (1/N)\Sigma_{j=1}^{N} \nabla J(\theta, x_j, y_j)$

$= (1/N)\Sigma_{j=1}^{N} \nabla J(\theta, x_j, y_j)$

$= g\_full$ ✓

Therefore: $E[g_B] = g\_full$ (unbiased!)

Variance (bonus):

$Var[g_B] = (1/m)Var[\nabla J(\theta, x_i, y_i)]$

As m increases:
- Variance decreases as 1/m
- Gradient becomes more accurate
- But computation increases linearly

Trade-off: Larger m → more accurate but slower

Q66. Calculate the number of gradient computations needed to process a dataset of N=10,000 samples for 50 epochs using:

- a) Batch GD
- b) SGD
- c) Mini-batch GD (batch size=32)

Answer:

a) Batch GD:

- One gradient computation per epoch (uses all N samples)
- Total: 50 epochs × 1 = **50 gradient computations**
- But each computation uses 10,000 samples!

b) SGD:

- One gradient per sample
- Total: 50 epochs × 10,000 samples = **500,000 gradient computations**

- Each computation uses 1 sample

c) Mini-batch (m=32):

- Batches per epoch: $\lfloor 10{,}000/32 \rfloor = 312$ (drop incomplete batch)
- Total: 50 epochs × 312 batches = **15,600 gradient computations**
- Each computation uses 32 samples

Comparison:

- Batch: Fewest computations, but each is expensive
- SGD: Most computations, but each is cheap
- Mini-batch: **Best balance** - moderate count, GPU-friendly

Wallclock time (typical):

- Batch: ~10 minutes/epoch → 500 minutes total
- SGD: ~5 minutes/epoch → 250 minutes total (parallelizes poorly)
- Mini-batch: ~2 minutes/epoch → **100 minutes total** ✓

Q67. For a neural network with L layers, each having n neurons, estimate:

- a) Number of parameters
- b) Forward pass computational complexity
- c) Backward pass computational complexity

Answer:

Assumptions: Fully connected, n neurons per layer

a) Parameters:

Each layer l:
Weights: $n \times n = n^2$
Biases: n
Total: $n^2 + n \approx n^2$ (for large n)

For L layers:
Total parameters $\approx L \times n^2$

b) Forward Pass Complexity:

Each layer:
  Matrix multiply: $O(n^2)$
  Activation: $O(n)$
  Total per layer: $O(n^2)$

For L layers:
  Total: $O(L \times n^2)$

c) Backward Pass Complexity:

Each layer (in reverse):
  Compute δ: $O(n^2)$ (matrix multiply with next layer)
  Compute gradients: $O(n^2)$
  Total per layer: $O(n^2)$

For L layers:
  Total: $O(L \times n^2)$

Note: Backprop same complexity as forward pass!

Example: L=5, n=1000

- Parameters: $5 \times 10^6 = 5$ million
- Forward: 5×10^6 operations
- Backward: 5×10^6 operations
- Training one sample: ~10 million operations

Conceptual Questions

Q68. Explain the difference between "optimization" and "learning" in machine learning. Can you have good optimization but poor learning?

Answer:

Optimization: Minimizing training loss

- Find $\theta^* = \text{argmin } L_{train}(\theta)$
- Mathematical problem
- Measures fit to training data

Learning: Minimizing expected loss on new data

- Want low $L_{test}(\theta)$ or $E[L(\theta)]$

- Statistical problem
- Measures generalization

Key Difference: Optimization ≠ Generalization

Yes, good optimization but poor learning:

Example 1 - Overfitting:

- Training loss: 0.01 (excellent optimization!)
- Test loss: 5.0 (terrible learning!)
- Model memorized training data

Example 2 - Wrong loss function:

- Optimizing squared error for classification
- Optimization succeeds (loss minimized)
- But task requires 0-1 loss (learning fails)

Example 3 - Distribution shift:

- Trained on images from camera A
- Test on images from camera B
- Optimization perfect on A
- Doesn't generalize to B

Solutions:

1. **Regularization**: Constrain optimization
2. **Early stopping**: Stop before overfitting
3. **Validation set**: Monitor learning during optimization
4. **Appropriate loss**: Use surrogate loss that aligns with true objective
5. **Data augmentation**: Make training distribution match test

Quote: "Optimization is what we can compute; learning is what we care about."

Q69. Why do we shuffle training data before each epoch in mini-batch SGD? What happens if we don't?

Answer:

Why shuffle:

1. **Remove sequential correlations**:
    - Data might have ordering (e.g., images sorted by class)
    - Consecutive batches would be too similar
    - Model sees biased view per batch
2. **Better gradient estimates**:
    - Random batches → more representative of full dataset
    - Unbiased gradient (proven earlier)
    - Explore loss landscape more uniformly
3. **Prevent catastrophic forgetting**:
    - Same order every epoch → model adapts to sequence
    - Early samples forgotten by time we see late samples
    - Shuffling ensures all data seen in different contexts
4. **Regularization effect**:
    - Different orderings provide slightly different optimization paths
    - Acts like noise injection (helps avoid sharp minima)

Without shuffling - Problems:

```python
# Bad: Data sorted by class
# Epoch 1: [class0, class0, ..., class1, class1, ...]
# Model updates heavily toward class0, then class1
# Final weights biased toward last-seen class
```

Specific issues:

- **Oscillating weights**: Weights flip between class preferences
- **Poor convergence**: Loss fluctuates wildly
- **Class imbalance amplified**: Last classes dominate
- **Longer training**: Need more epochs to converge

Implementation:

```python
```

```
for epoch in range(num_epochs):
    indices = np.random.permutation(N)  # Shuffle!
    for batch_start in range(0, N, batch_size):
        batch_idx = indices[batch_start:batch_start+batch_size]
        batch = data[batch_idx]
        # Train on batch...
```

Exception: When NOT to shuffle:

- **Time series**: Future shouldn't inform past
- **Sequential data**: Order matters (RNNs sometimes)
- But even then, shuffle sequences (not within sequence)

Q70. What is the "dying ReLU" problem? How does it happen and how can it be prevented?

Answer:

Problem: Neurons get stuck outputting zero for ALL inputs, never recover.

Mechanism:

1. **During training**: Some neuron has large negative bias

$$z = Wx + b < 0 \text{ for all } x \text{ in dataset}$$
$$a = \text{ReLU}(z) = 0 \text{ for all } x$$

2. **Gradient**:

$$\partial L/\partial W = \partial L/\partial a \times \partial a/\partial z \times \partial z/\partial W$$
$$= \partial L/\partial a \times 0 \times x \quad [\text{since } \partial \text{ReLU}/\partial z = 0 \text{ when } z<0]$$
$$= 0$$

3. **No learning**: W and b never update (zero gradient)
4. **Permanent death**: Neuron stays dead forever

How it happens:

1. **Bad initialization**: W, b initialized such that most $z<0$
2. **Large learning rate**: Big update pushes W to very negative values
3. **Data distribution**: Most training examples give negative pre-activations
4. **Explosion during training**: Gradient spike causes catastrophic update

Example:

Before: W = [0.5, -0.3], b = 0.1
Big gradient update: W := W - 10.0 × [large gradient]
After: W = [-50.2, -100.3], b = -75.4
Now: Wx + b < -75 for all reasonable x → DEAD

Prevention:

1. **Use Leaky ReLU**:

$f(z) = \max(0.01z, z)$
$f'(z) = 0.01$ (if $z<0$) or 1 (if $z>0$)

Always has gradient → can recover!

2. **Parametric ReLU (PReLU)**:

$f(z) = \max(\alpha z, z)$ where α is learnable

3. **ELU (Exponential Linear Unit)**:

$f(z) = z$ (if $z>0$) or $\alpha(e^z-1)$ (if $z<0$)

Smooth, non-zero gradient everywhere

4. **Proper initialization**:
    - He initialization for ReLU: $W \sim N(0, 2/n\_in)$
    - Keeps activations in good range
5. **Smaller learning rate**:
    - Prevents catastrophic updates
    - Use adaptive methods (Adam)
6. **Batch Normalization**:
    - Keeps pre-activations centered
    - Reduces extreme negative values

Monitoring:

```python
def check_dead_neurons(model, data):
    activations = model.get_activations(data)
    for layer, act in enumerate(activations):
        dead = (act == 0).all(axis=0).sum()
        print(f"Layer {layer}: {dead}/{act.shape[1]} dead neurons")
```

Healthy network: <10% dead neurons **Problem**: >30% dead neurons

Scenario-Based Questions

Q71. You're training two models on the same data:

- Model A (100 parameters): Train loss=0.5, Test loss=0.6
- Model B (10,000 parameters): Train loss=0.1, Test loss=2.0

Which model is better? Explain your reasoning considering bias-variance tradeoff.

Answer:

Model A is better (lower test loss)